

SDOQAP — เนื้อหาสไลด์แบบเล่าเรื่อง (10 สไลด์)

แหล่งข้อเท็จจริงของระบบ: [system-actual-analysis-topics-2-6.md](#) (ผลวิเคราะห์โค้ด) — ไม่ได้อ่านโค้ดใหม่ **Pattern** การเล่าเรื่อง: ไฟล์ AI ETL Introduction.pdf (7 หน้า)
· ไม่ใช่รายงาน White Box และไม่เรียงตามเกณฑ์ ท้ายทุกสไลด์ระบุว่าเนื้อหานี้ครอบคลุมเกณฑ์ข้อใด (ไม่ใช่เลขเกณฑ์เป็นชื่อสไลด์)

สถานะที่ใช้บอกความจริงของแต่ละความสามารถ: ● ทำงานจริง · ● ทำงานบางส่วน/มีข้อจำกัด · ● มีแต่ UI · ● ข้อมูลจำลอง · ● ไม่มีในระบบ

Pattern ที่ถอดจากไฟล์ AI ETL Introduction

องค์ประกอบ	ไฟล์อ้างอิงทำอะไร	นำมาใช้กับ SDOQAP อย่างไร
หัวข้อสไลด์เป็นคำถามหรือประโยคเล่าเรื่อง	"AI ETL คืออะไร?", "แล้วข้อมูลเข้าสู่ Data Pipeline จากไหน?", "เมื่อข้อมูลเปลี่ยน Traditional ETL ก็เริ่มมีข้อจำกัด"	ตั้งชื่อสไลด์เป็นคำถามที่คนฟังสงสัยจริง แล้วให้สไลด์ตอบ
เล่าเป็นลำดับ ภาพใหญ่ → ปัญหา → วิธีแก้ → ตัวอย่างจริง	ETL คืออะไร → ข้อจำกัด 4 ข้อ → AI ETL → AI ช่วยตรงไหน 5 จุด → แหล่งข้อมูลจริง	Slide 1–2 วางปัญหา · 3–7 เดินตามท่อ · 8–9 ผลลัพธ์/AI · 10 สรุป
การ์ดเลข 01–05 ประโยคเดียวต่อใบ	หัวข้อสั้นๆ + คำอธิบายสั้น 1–2 บรรทัด	ใช้ 4–5 การ์ดต่อสไลด์ ไม่เกินนี้
เทียบ 2 ฝั่ง (Traditional vs AI)	ตาราง/กล่องคู่ขนาน	ใช้ "ปัญหา → สิ่งที่ SDOQAP ทำ" และ "ส่วนที่เป็น AI vs ส่วนที่เป็นกฎ/สถิติ"
แถบสรุปล่างสไลด์	ประโยคเดียวที่คนฟังควรจำ ("ความหลากหลายของแหล่งข้อมูล... ต้องยืดหยุ่น")	ทุกสไลด์มี "ประโยคจำ" 1 ประโยค
ประโยคเชื่อมโยงไปสไลด์ถัดไปเป็นคำถาม	"แล้วเราจะนำแนวคิด AI ETL มาสร้างเป็น Product ได้อย่างไร?"	ทุกสไลด์ปิดด้วย "ประโยคเชื่อม"
จังหวะสีพื้นหลัง	สไลด์สว่าง (เนื้อหา) สลับสไลด์มืด (จุดเปลี่ยนเรื่อง)	พื้นมืดที่ Slide 2 (จุดเปลี่ยนเข้าท่อ) และ Slide 9 (AI) นอกนั้นสว่าง
ระดับรายละเอียด	ไม่ลงโค้ด ใช้คำบรรยายพร้อมไอคอน ตัวอย่างจริงเป็นชื่อชุดข้อมูล	บนสไลด์ไม่มีชื่อไฟล์โค้ด — เก็บ evidence ไว้ในหมายเหตุผู้พูด

หมายเหตุเรื่องชุดข้อมูลที่ไฟล์อ้างอิงระบุ: หน้า 7 ของไฟล์ระบุชุดทดสอบ Online Retail II, Road/Traffic Accident, NYC 311, Socrata, Data.go.th, World Bank API, Reddit Comments, Pagila, AdventureWorks ผมตรวจใน repo แล้ว พบเฉพาะ **Olist (ใน n8n workflow)** และ **data.go.th (resolver ใน API)** ส่วนชื่ออื่นไม่พบในโค้ด/ข้อมูลของ repo — ถ้าจะพูดว่า "ทดสอบกับชุดเหล่านี้" ต้องมีหลักฐานการรันของทีมงาน เอกสารนี้จึงใช้เฉพาะชุดที่เห็นใน repo (ข้อมูลทดสอบ data/evaluation/ ขนาด 10,000 / 100,000 / 500,000 / 1,000,000 แถว และตัวอย่างใน data/samples/ — ยังไม่ได้ตรวจผลการรันกับไฟล์ขนาดใหญ่)

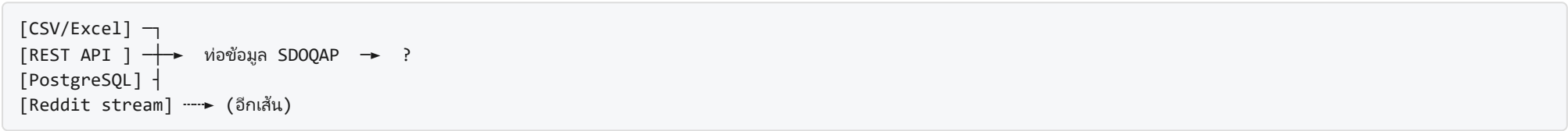
Slide 1 — "ข้อมูลของเรามาจากไหน?"

Key Message: ข้อมูลจริงไม่ได้มาจากที่เดียวหรือหน้าตาเดียว — ระบบต้องรับได้หลายแบบ และต้องรู้ว่าแต่ละแบบเสี่ยงอะไร

เนื้อหาบนสไลด์

- หัวใหญ่: ข้อมูลของเรามาจากไหน? · รองหัว: ข้อมูลกระจายอยู่ในหลายรูปแบบและหลายระบบ
- 4 การ์ด (เลข 01–04) ตามที่ระบบรับเข้าได้จริง:
 - 01 ไฟล์ CSV / Excel ● — อัปโหลดผ่านหน้าเว็บ (Excel อ่าน sheet แรก)
 - 02 REST API ● — ดึง JSON/CSV จาก URL (มีตัวช่วยสำหรับ data.go.th)
 - 03 ฐานข้อมูล PostgreSQL ● — เขียน SELECT ดึงแบบอ่านอย่างเดียว
 - 04 Stream (Reddit → Kafka) ● — มีจริงแต่เป็นอีกเส้นทางหนึ่ง ยังไม่ผ่านการตรวจคุณภาพ
- แถบสรุปล่าง: "ต่างแหล่ง ต่างรูปแบบ ต่างความเสี่ยง — ท่อข้อมูลจึงต้องจัดการได้ยืดหยุ่น"
- ข้อมูลทดสอบที่ใช้ในระบบนี้ (ติดป้ายเล็ก): ชุดคะแนนนักเรียน 10,100 แถว (ใส่ข้อผิดพลาดไว้ล่วงหน้า), ไฟล์ขนาด 10K–1M แถวใน repo

Diagram ที่ควรรวด (สไลด์เดียวกับหน้า 7 ของไฟล์อ้างอิง: การ์ดซ้อนแนวตั้ง ไอคอนซ้าย คำอธิบายขวา)



การ์ดที่ 4 ใช้เส้นประและป้ายสีเหลือง "ยังไม่เชื่อมระบบตรวจคุณภาพ"

สิ่งที่ผู้พูดควรพูด (~45 วินาที) "ก่อนจะพูดถึงท่อข้อมูล ต้องเริ่มจากคำถามว่าข้อมูลมาจากไหน ในระบบของเรา ข้อมูลมาได้ 4 ทาง ไฟล์ CSV หรือ Excel ที่ผู้ใช้อัปโหลด เว็บเซอร์วิส REST API ฐานข้อมูล PostgreSQL และสตรีมจาก Reddit สามทางแรกต่อเข้าท่อตรวจคุณภาพของเราครบ ส่วน Reddit เป็นเส้นทางแยกที่ยังไม่ผ่านการตรวจ แต่ละทางมีความเสี่ยงต่างกัน ไฟล์อาจเข้ารหัส ผิด API อาจมีข้อมูลซ้อนกัน ฐานข้อมูลอาจใหญ่เกินไป ดังนั้นเราต้องมีท่อที่จัดการได้ยืดหยุ่น"

Technical evidence จากโค้ด

- ไฟล์: API/pipeline.py:315-333 (ingest_csv ; Excel → CSV ที่ :324-329)
- REST: pipeline.py:335-508 (ingest_api ; resolver data.go.th :354-435)
- PostgreSQL: pipeline.py:567-613 (ingest_rdbms ; db_type ต้องเป็น postgresql)
- Stream: streaming_job.py + reddit_stream.py → Parquet /data/reddit/parquet และ ES reddit ; ไม่มีการอ้างถึงใน spark_quality_engine.py (ยืนยันด้วย grep reddit ใน engine ไม่พบ)
- ข้อควรระวัง: แท็บ "API" และ "Stream" บนหน้า Ingestion เป็นโหมตสาริต (whitebox.py:1593-1618 ตอบ "simulated": True) — ถ้าจะสาธิตการดึงจาก API ให้ใช้เรียก API ตรงหรือผ่าน n8n ไม่ใช่กดแท็บนั้น
- ไม่รองรับ: ไฟล์ JSON/Parquet/Avro/XML, ฐานข้อมูลอื่นนอกจาก PostgreSQL ●

ประโยคจำ: ความหลากหลายของแหล่งข้อมูลคือเหตุผลที่ต้องมีท่อข้อมูล ประโยคเชื่อม: "แล้วทำไมแค่ดึงข้อมูลมาเก็บไว้ถึงไม่พอ?"

ครอบคลุมเกณฑ์: ข้อ 3 (ส่วนแหล่งข้อมูลที่รองรับ) · เป็นบริบทให้ข้อ 2

Slide 2 — "ทำไมต้องมี Data Pipeline หลายกระบวนการ?" (พื้นมืด — จุดเปลี่ยนเข้าสู่ท่อ)

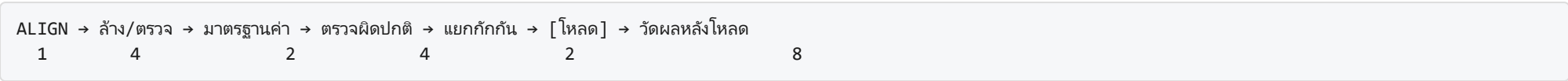
Key Message: ข้อมูลดิบมีปัญหาหลายชั้น แต่ละชั้นต้องการกระบวนการเฉพาะ — ระบบจึงมี 21 ชั้นใน 3 ช่วง จัดเป็น 6 กลุ่ม

เนื้อหาบนสไลด์ (ใช้โครงเดียวกับหน้า 3 ของไฟล์อ้างอิง: 4 ปัญหาเป็นแถวเลข 01–04 พร้อมคำตอบ)

#	เมื่อข้อมูลดิบมาแบบนี้...	ระบบตอบด้วยกระบวนการ...
01	โครงสร้างไม่ตรงกัน — ชื่อ/ชนิดคอลัมน์ต่างจากที่คาด	จัดโครงสร้าง (ALIGN) + ตรวจสอบการเปลี่ยนแปลง schema (schema drift)
02	ค่าผิด ว่าง หรือซ้ำ	ล้างข้อมูล · ตรวจสอบ · ลบซ้ำ
03	รูปแบบไม่เหมือนกัน — เช่น วันที่หลายแบบ รวมปี พ.ศ.	มาตรฐานค่า (standardize)
04	ค่าผิดปกติที่ไม่ได้ผิดกฎตรง ๆ	ตรวจด้วยสถิติ (IQR / Z-score) แล้วแยกเข้ากักกัน
05	ข้อมูลเสียไม่ควรปนกับข้อมูลดี	แยก Valid / Invalid → Quarantine
06	ต้องรู้ว่าผลเป็นอย่างไร	วัดผลหลังไหลด (คะแนนคุณภาพ, ความสดใหม่ ฯลฯ)

- แถบ **21 = 1 + 12 + 8** (ALIGN 1 · TRANSFORM 12 · POST-LOAD 8) พร้อมสัญลักษณ์ "+" ขั้นตอนภายในอีก 7 + กระบวนการฝัง API อีกอย่างน้อย 10"
- แถบสรุปล่าง: "ทุกชั้นรันอัตโนมัติ หลังผู้ใช้ส่งข้อมูลเข้าระบบ"

Diagram ที่ควรรวาท — แถบ 6 ช่องต่อกัน (แนวนอน) ตัวเลขใต้ช่องคือจำนวน stage



ช่อง [โหลด] วาดเส้นประ (เป็นขั้นตอนภายใน ไม่ใช่ stage ที่ลงทะเบียน)

สิ่งที่ผู้พูดควรพูด (~55 วินาที) "ถ้าข้อมูลสะอาดอยู่แล้ว เราคงไม่ต้องมีท้อ แต่ข้อมูลจริงมีปัญหาหลายชั้น โครงสร้างไม่ตรง ค่าผิด ค่าว่าง ซ้ำ รูปแบบไม่เหมือนกัน และมีค่าที่ผิดปกติแบบที่กฎตรง ๆ จับไม่ได้ แต่ปัญหาต้องใช้กระบวนการต่างกัน ระบบเราจึงแบ่งเป็น 21 ชั้น ใน 3 ช่วง คือจัดโครงสร้าง 1 ชั้น แปลงและตรวจ 12 ชั้น และวัดผลหลังโหลด 8 ชั้น เพื่อให้เห็นภาพ ผมจัดเป็น 6 กลุ่ม นอกจากนี้ยังมีขั้นตอนที่อยู่ในโค้ดอีก เช่น การล็อกตารางและการ MERGE และกระบวนการฝัง API เช่น ตรวจไฟล์ซ้ำ ทั้งหมดทำงานอัตโนมัติ"

Technical evidence จากโค้ด

- รายชื่อ 3 ช่วง: S/pipeline/plan.py:2-6 ; ลงทะเบียนและจับเวลาต่อ stage: registry.py:17-32 (stage_seconds)
- ลำดับจริงของงานหนึ่งรอบ: SQE:run_quality_check (spark_quality_engine.py:1551-1811)
- ขั้นตอนภายใน 7: ขอ lock · อ่าน raw · เขียน quarantine · MERGE active · OPTIMIZE/VACUUM · trigger Gold · archive raw
- คิวงาน 1 งาน/ตาราง: trigger_core.py:53-82 ; lock ใน ES พร้อม heartbeat: SQE:126-218
- **ที่ต้องบอกถ้าถูกถาม:** standardize_categories ต้องมี standardization_rules ใน registry ซึ่งไม่พบตัวเขียนคำนี้ใน repo (ผลน้อย) 🟡 ; anomaly_induced มักว่าง (ต้องมีกฎที่คนอนุมัติ) 🟡

ประโยคจ่า: หนึ่งปัญหา หนึ่งกระบวนการ — ยิ่งข้อมูลดิบ ยิ่งต้องมีหลายด่าน **ประโยคเชื่อม:** "ด่านแรกสุดคือก่อนข้อมูลจะเข้าระบบ — ระบบตรวจอะไรบ้าง?"

ครอบคลุมเกณฑ์: ข้อ 2 (หลัก)

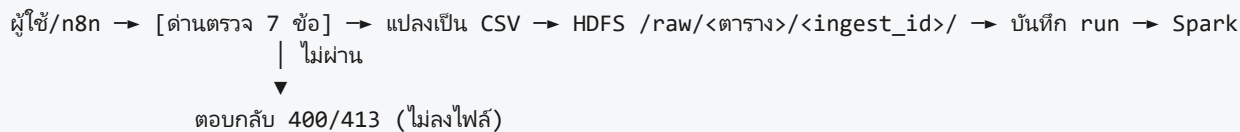
Slide 3 — "ก่อนรับข้อมูลเข้าระบบ เราตรวจอะไร?"

Key Message: ด้านแรกกันของเสียและของอันตรายตั้งแต่ประตู แล้วเก็บสำเนาดิบพร้อมบันทึกประวัติ ทุกครั้งที่นำเข้า

เนื้อหาบนสไลด์

- 7 การ์ด (ตรวจที่ API ก่อนลงไฟล์) ทั้งหมด ● :
 - **01 ยืนยันตัวตน** (session หรือ service key)
 - **02 ชื่อตาราง** — อนุญาตเฉพาะตัวอักษร ตัวเลข `_` `-` (กั้นการแอบอ้างเส้นทางไฟล์)
 - **03 ขนาดไฟล์** — ไม่เกิน 200 MB
 - **04 ไฟล์ว่าง** — ปฏิเสธทันที
 - **05 ไฟล์ซ้ำ** — คำนวณ SHA-256 ถ้าเคยรับแล้วจะไม่รับซ้ำ
 - **06 คอลัมน์คีย์หลัก** — ถ้าตารางมีคีย์ลงทะเบียนไว้ ไฟล์ต้องมีคอลัมน์นั้น
 - **07 ป้องกัน URL/SQL อันตราย** — REST: อนุญาตเฉพาะ host ที่กำหนด · ฐานข้อมูล: SELECT เท่านั้น, อ่านอย่างเดียว, timeout 30 วินาที, ไม่เกิน 1,000,000 แถว
- ช่องล่าง "แล้วหลังจากผ่านด่าน": แปลงทุกแหล่งเป็น CSV → เก็บ **ไฟล์ดิบ 1 โพลเดอร์ต่อ 1 การนำเข้า** ใน HDFS → บันทึก run (สถานะ QUEUED) → ส่ง Spark
- กล้อง "สิ่งที่ยังไม่มี" (พูดเอง ไม่รอถูกถาม): ● ไม่เทียบจำนวนแถวต้นทางกับที่รับเข้า · ● ไม่ตรวจ encoding/delimiter · ● REST ฝั่ง API ไม่มี pagination/retry
- แถบสรุปล่าง: "รับเข้าอย่างมีบันทึก — รู้ว่าจะไรเข้ามา เมื่อไร และมาจากไหน"

Diagram ที่ควรรวด



ได้ "บันทึก run" วาดการ์ดเล็กแสดงสถานะ: QUEUED → RUNNING → SUCCEEDED / FAILED / SKIPPED / TRIGGER_FAILED

สิ่งที่ผู้พูดควรพูด (~55 วินาที) "ก่อนข้อมูลเข้าระบบ เราตรวจ 7 อย่าง ตั้งแต่ใครส่งมา ชื่อตารางปลอดภัยไหม ไฟล์ใหญ่เกิน 200 เมกะไบต์หรือเปล่า ว่างไหม และเคยส่งมาแล้วหรือยังโดยใช้ SHA-256 ถ้าเป็น REST API จะอนุญาตเฉพาะเว็บที่กำหนด ถ้าเป็นฐานข้อมูลจะอนุญาตเฉพาะคำสั่ง SELECT แบบอ่านอย่างเดียว พอผ่านแล้ว ทุกแหล่งถูกแปลงเป็น CSV เก็บใน HDFS แยกโพลเดอร์ต่อการนำเข้า และบันทึกสถานะลง Elasticsearch ผมขอบอกตรง ๆ ว่า ตอนนี้ระบบยังไม่ได้เทียบจำนวนแถวต้นทางกับที่รับเข้า และยังไม่ตรวจ encoding ของไฟล์"

Technical evidence จากโค้ด

- ขั้นตอนทั้งหมด: API/pipeline.py:271-303 (land_and_queue), :315-333 (ingest_csv)
- ชื่อตาราง: validation.py:4-18 (regex `^[A-Za-z0-9_-]{1,128}$`) · ขนาด: ingest_guards.py:96-109 (MAX_UPLOAD_MB ค่าเริ่มต้น 200)
- SHA-256 + สถานะ: run_registry.py:21-65 (เข้าเฉพาะสถานะ QUEUED/RUNNING/SUCCEEDED; FAILED/SKIPPED/TRIGGER_FAILED อัปโหลดซ้ำได้)
- SSRF/SQL guard: ingest_guards.py:22-93
- เขียน HDFS: pipeline.py:194-227 (retry 5 ครั้ง ห่าง 3 วินาที), path `/data/raw/<table>/<ingest_id>/<table>.csv`

- ถ้าผิดพลาด: HTTP 400/413; เขียน HDFS ล้ม → 500; daemon ติดต่อไม่ได้ → 503 + `TRIGGER_FAILED` ไปด้วย `POST /pipeline/retry/<ingest_id>` (`pipeline.py:155-186, 291-295`)
- ข้อจำกัด: record count ต้นทาง-ที่รับ ไม่มี (บันทึกแค่ `size_bytes`); n8n ที่ตั้งเวลาไว้เขียน HDFS เส้นแบนโดยไม่ผ่านการตรวจ checksum
- เส้นทาง n8n ตั้งเวลา: Olist CSV ตายตัว, data.go.th `limit=10000, SELECT * FROM sales_records` (`ingestion_workflow.json:119, 194, 274`) ● แหล่งตายตัวสำหรับเดโม

ประโยคจา: ผ่านด่านก่อน จึงได้เข้าท่อ และทุกครั้งที่เข้ามีบันทึก **ประโยคเชื่อม:** "ข้อมูลเข้ามาแล้ว แต่แต่ละแหล่งหน้าตาไม่เหมือนกัน — จะทำให้ใช้ร่วมกันได้อย่างไร?"

ครอบคลุมเกณฑ์: ข้อ 3 (หลัก) · ข้อ 2

Slide 4 — "ข้อมูลต่างรูปแบบ ทำให้ใช้ร่วมกันได้อย่างไร?"

Key Message: Spark อ่านทุกอย่างเป็นข้อความก่อน แล้วค่อยแปลงตามโครงสร้างที่ลงทะเบียนไว้ — ผลคือข้อมูลจากหลายแหล่งพูด "ภาษาเดียวกัน"

เนื้อหาบนสไลด์

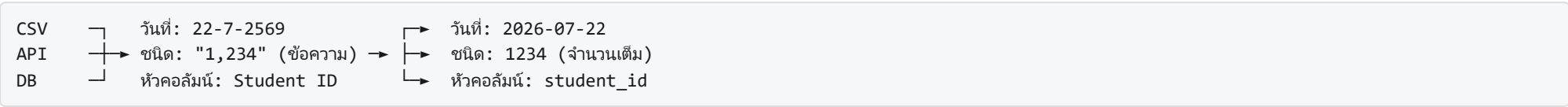
- 5 การ์ด Transformation ที่แท้จริง (●):
 - **01 แปลงชนิดข้อมูล** — `"1,234"` → `1234`; วันที่หลายแบบ → timestamp
 - **02 จัดชื่อคอลัมน์ให้ตรงกัน** — `Student ID` → `student_id` (จับคู่แบบ fuzzy กับชื่อใน registry — เป็นอัลกอริทึมไม่ใช่ AI)
 - **03 มาตรฐานวันที่ (รวม พ.ศ.→ค.ศ.)** — `22-7-2569` → `2026-07-22`
 - **04 ลบแถวซ้ำตามคีย์หลัก** — ซ้ำ 2 ชั้น
 - **05 รับมือโครงสร้างเปลี่ยน** — คอลัมน์ใหม่ถูกเพิ่ม · คอลัมน์ที่หายถูกเติม NULL พร้อม alert · ทุกการเปลี่ยนบันทึกเป็นข้อเสนอรอคนอนุมัติ
- ตาราง Before → After (สไลด์ตารางเทียบของไฟล์อ้างอิง):

ก่อน (มาจากหลายแหล่ง)	หลังแปลง
<code>22-7-2569 · 20/07/2026 · 14 Jul 2026</code>	<code>2026-07-22 · 2026-07-20 · 2026-07-14</code>
<code>"1,234"</code> (ข้อความ)	<code>1234</code> (จำนวนเต็ม)
<code>Student ID</code>	<code>student_id</code>

- ● ที่ทำได้แต่ต้องตั้งกฎเอง: เติมค่าว่าง (`fillna / calculate`), จับคู่ความหมายแบบ semantic (threshold 0.85), map คำเป็นหมวด

- ● ที่ไม่มี: join หลายตารางใน Spark path, เติมค่าว่างด้วย mean/median/mode, regex เป็นกฎของผู้ใช้
- แถบสรุปล่าง: "อ่านเป็นข้อความก่อน แล้วแปลงตามโครงสร้างที่ตกลงไว้ — ไม่ปล่อยให้ Spark เดาชนิดเอง"

Diagram ที่ควรรวด — ภาพ "หลายรูปแบบเข้า → รูปแบบเดียวออก" (กรวย)



สิ่งที่ผู้พูดควรพูด (~55 วินาที) "พอข้อมูลเข้ามา แต่ละแหล่งหน้าตาต่างกัน เช่น วันที่เขียนไม่เหมือนกัน บางแหล่งเป็นปี พ.ศ. ตัวเลขมาเป็นข้อความที่มีลูกน้ำ ชื่อคอลัมน์เขียนไม่เหมือนกัน Spark จึงอ่านทุกอย่างเป็นข้อความก่อน แล้วแปลงตามโครงสร้างที่ลงทะเบียนไว้ วันที่ทุกแบบถูกแปลงเป็นรูปแบบเดียว ตัวเลขถูกแปลงเป็นเลข ชื่อคอลัมน์ถูกจัดให้ตรงกับที่ลงทะเบียน แกว่เข้าถูกตัดตามคีย์หลัก ถ้าโครงสร้างต้นทางเปลี่ยน ระบบตรวจพบและบันทึกเป็นข้อเสนอให้คนอนุมัติ ข้อควรบอกคือ ชั้นลบซ้ำชั้นแรกยังไม่นับรวมในยอดที่ถูกกักกัน"

Technical evidence จากโค้ด

- แปลงชนิด/จัดชื่อ: S/stages/schema.py:10-98 (Integer = ตัดอักขระที่ไม่ใช่ตัวเลขแล้ว cast :64-68; Timestamp = 6 รูปแบบ + epoch :73-93; rename ด้วย normalize_name :28-35, names.py:4-15)
- วันที่ + พ.ศ.: standardize.py:6-57 (ปี > 2500 ลบ 543 :31-32, 46-47) — ตัวอย่างจาก unit test test_stage_cleansing.py:40-43, ชุด services/spark/grocery_sales.csv
- ตัวอย่างชนิดข้อมูล: test_stage_schema.py:6-14; ตัวอย่างลบซ้ำ: test_stage_cleansing.py:8-12; ชุด services/spark/users.csv
- Schema drift: schema.py:101-243 — บันทึก sdoqap_schema_drifts / sdoqap_schema_proposals; อนุมัติผ่าน API/schema.py:63-183; **PENDING** ไม่ได้หยุดข้อมูล (คอลัมน์ใหม่ถูกเพิ่มแล้ว และ Delta autoMerge ขยายตาราง SQE:85, schema.py:146-148)
- ลบซ้ำ 2 ชั้น: cleansing.py:6-44 (ชั้นแรก ไม่นับ) และ :118-138 (ชั้นสอง ส่ง quarantine duplicate_records)
- ผู้กำหนด logic: Dev (อัลกอริทึม) · registry/อนุมานจากข้อมูล (System) · DSL (User/LLM)
- ข้อจำกัดที่ยืนยันจากการอ่านโค้ด (ยังไม่ได้รัน): วันที่ใช้กับคอลัมน์ชื่อ วันที่ / date / Date เท่านั้น ไม่ตรวจปฏิทิน (31-02 ผ่าน); regex Integer ตัด "." ทำให้ "12.00" อาจกลายเป็น 1200 (schema.py:64-68)
- Semantic standardize: คะแนน = 0.4·substring + 0.3·token-overlap + 0.3·bigram-cosine (S/semantic/similarity.py:35-69) — เป็นสูตรความคล้าย ไม่ใช่ LLM

ประโยคจำ: ต่างที่มา แต่ออกมาเป็นภาษาเดียวกัน ประโยคเชื่อม: "ข้อมูลที่แปลงแล้วใช่ว่าจะดี — เราตรวจคุณภาพอย่างไร?"

ครอบคลุมเกณฑ์: ข้อ 4 (หลัก) · ข้อ 2

Slide 5 — "เราตรวจคุณภาพข้อมูลอย่างไร?"

Key Message: ตรวจ 3 ชั้น — กฎตรง ๆ, สถิติ, และการเฝ้าดูแนวโน้ม — และทุกแถวที่ไม่ผ่านมี "เหตุผลระบุได้"

เนื้อหาบนสไลด์

- **ชั้นที่ 1 กฎตรง ๆ (ทุกข้อ → เหตุผลบันทึกต่อแถว)** ●
 - คีย์หลักว่าง → `missing_primary_key`
 - ค่าว่างในคอลัมน์ → `null_value_in_<คอลัมน์>`
 - ชนิดไม่ถูกต้อง → `invalid_type_<คอลัมน์>`
 - วันที่ว่าง → `missing_date`
 - คีย์หลักซ้ำ → `duplicate_records`
 - เกินช่วงที่ตั้ง → `out_of_range_<คอลัมน์>` (ต้องตั้ง `range_checks`)
- **ชั้นที่ 2 สถิติ** ● — IQR (ค่าเริ่มต้นตัวคูณ 1.5) และ Z-score เกิน 3.0 → ตรวจทุกคอลัมน์ตัวเลข
- **ชั้นที่ 3 เฝ้าดูแนวโน้ม** ● — คะแนนต่ำกว่าเกณฑ์ → alert; อัตราแถวที่ผิดปกติเทียบกับประวัติ (ต้องมี ≥ 3 รอบ) → alert; profile drift (PSI) เมื่อเปิด AI advisor
- **สูตรคะแนน:** $\text{Quality Score} = \text{Clean} \div (\text{Clean} + \text{Quarantine}) \times 100$
- ตัวอย่างเหตุผลจริง (กล่องเล็ก) ชุดคะแนนนักเรียน 10,100 แถว ที่ใส่ข้อผิดพลาดล่วงหน้า: คะแนนว่าง 300 · คะแนนเกินช่วง 200 · ค่าผิดปกติ 100 · แถวซ้ำ 100 (ตามไฟล์ `ground_truth.csv` และ README ของชุดข้อมูล — ตัวเลขนี้เป็นเฉลยที่ฝังไว้ ไม่ใช่ผลการรัน)
- มิติที่วัดจริง: ความครบ (Completeness) · ความไม่ซ้ำ (Uniqueness) · ความถูกต้องตามกฎ (Validity) ● · ความสดใหม่ (Timeliness) ● วัดแต่ไม่เทียบเกณฑ์ · ความสอดคล้อง/ความแม่นยำ ● ในเส้นหลักยังไม่มี
- แถบสรุปล่าง: "ไม่มีแถวไหนถูกปฏิเสธโดยไม่บอกเหตุผล"

Diagram ที่ควรรว — พิจารณา/บันได 3 ชั้น (กฎ → สถิติ → แนวโน้ม) ข้างขวามีกล่องสูตรคะแนน และตัวอย่างตาราง quarantine (คอลัมน์: ข้อมูลเดิม | `reject_reason`)

สิ่งที่ผู้พูดควรพูด (~55 วินาที) "เราตรวจคุณภาพ 3 ชั้น ชั้นแรกคือกฎตรง ๆ เช่น คีย์หลักห้ามว่าง ห้ามมีค่าว่าง ชนิดต้องถูก ห้ามซ้ำ ชั้นที่สองใช้สถิติ IQR และ Z-score จับค่าที่ผิดปกติแม้จะไม่ผิดกฎตรง ๆ ชั้นที่สามเฝ้าดูแนวโน้ม ถ้าคะแนนตกหรืออัตราแถวเสียพุ่งผิดปกติ ระบบแจ้งเตือน ทุกแถวที่ไม่ผ่านจะมีเหตุผลกำกับ คะแนนคุณภาพคือสัดส่วนแถวที่ผ่านต่อแถวทั้งหมด ข้อควรรู้คือมิติความสอดคล้องและความแม่นยำยังไม่มีในเส้นหลัก และบางสวิตช์ในไฟล์ตั้งค่า เช่น ปิดการตรวจคีย์ว่าง ยังไม่ถูกอ่านโดยระบบ"

Technical evidence จากโค้ด

- กฎ: `S/stages/cleansing.py:47-135` · ช่วงค่า `rules.py:15-33` · IQR/Z-score/induced `anomaly.py:6-118` (Z-score ฮาร์ดโค้ด 3.0 ที่ `:58`; IQR รันเมื่อ `value_range.mode` เป็น `auto/adaptive` :15-17)

- สูตรคะแนน `metrics.py:168-175`; `alert` :177-183; Z-score อัตรา quarantine :184-210
- ความสดใหม่ `metrics.py:119-158` (`max_lag_hours`) — `freshness_threshold_hours` ใน `rules` ถูกบันทึกแต่ไม่นำมาเทียบ
- **สวิตช์ที่ไม่ถูกอ่านโดย engine** 🟡: `null_primary_key`, `duplicate_check`, `null_date_column`, `null_checks`, `value_range.column_overrides` — แก้ในไฟล์/หน้า `config` แล้วพฤติกรรมไม่เปลี่ยน
- ตัวอย่างข้อผิดพลาดของกฎเริ่มต้นที่ควรรู้: ใน `rules_config.json:596-608` กฎ `fillna score=0` มาก่อน `calculate ... IS NULL` ทำให้เงื่อนไขหลังไม่ทำงาน (แถวคะแนนว่างกลายเป็น 0 แล้วผ่านเข้า `active`) — เป็นข้อสรุปจากการอ่านโค้ด ยังไม่ได้รับ
- ข้อมูลทดสอบ: `data/evaluation/README.md`, `student_course_score_evaluation_dataset.zip` (`dirty_dataset.csv` + `ground_truth.csv`)

ประโยคจำ: ทุกแถวที่ถูกกัก มีเหตุผล **ประโยคเชื่อม:** "แล้วเมื่อข้อมูลผ่านหรือไม่ผ่าน ระบบทำอะไรต่อ?"

ครอบคลุมเกณฑ์: ข้อ 4 (กฎการตรวจ/คุณภาพ) · ข้อ 2

Slide 6 — "ข้อมูลผ่าน / ไม่ผ่าน จัดการอย่างไร?"

Key Message: ระบบ แยก ข้อมูลที่ไม่ผ่านออกก่อนโหลด (ไม่ทิ้ง) ส่วน **คะแนนคุณภาพ** คิดหลังโหลดเพื่อแจ้งเตือนและควบคุมขั้นต่อไป ไม่ได้หยุดการโหลด

เนื้อหาบนสไลด์

- สองด้านที่ต้องแยกให้ชัด (ใช้การเทียบสองฝั่งแบบหน้า 4 ของไฟล์อ้างอิง):

ด้านที่ 1 · แยกรายแถว (ก่อนโหลด) 🟢	ด้านที่ 2 · คะแนนคุณภาพ (หลังโหลด)
แถวผ่าน → ไปตาราง Active	คำนวณหลังเขียน Delta แล้ว
แถวไม่ผ่าน → ไปตาราง Quarantine พร้อมเหตุผล	ใช้ส่ง alert · ติดตามสถานะ <code>warnings</code> · ตัดสินว่าจะสร้างตารางสรุป Gold หรือไม่
ไม่ลบทิ้ง	ไม่หยุดการโหลด (🔴 ไม่มี gate ระดับตารางก่อนโหลด)

- **เกณฑ์ (threshold):** ค่าเริ่มต้น 90 พ้น 70 · โหมด `adaptive` = เฉลี่ย 15 รอบล่าสุดลบส่วนเบี่ยงเบน แต่ไม่ต่ำกว่าพื้น
- ผู้ใช้ปลายทางถามได้ว่า "ตารางนี้ใช้ได้ไหม": `Trust-check` → `SAFE` / `WARNING_SUSPECT` / `HALT_INGEST` 🟢 (คำแนะนำ ผู้เรียกเลือกใช้เอง)
- หลังกักกัน: มีทางไปต่อ 2 ทาง → (ก) AI เสนอกฎ ให้คนอนุมัติ (ข) ระบบแก้อัตโนมัติรอบเดียว (ดู Slide 9)
- แถบสรุปล่าง: "ข้อมูลเสียไม่ปนกับข้อมูลดี และไม่หายไปไหน"

Diagram ที่ควรรว



ใส่กรอบแดงรอบ "คิดคะแนนหลังโหลด" พร้อมข้อความ "ไม่หยุดการโหลด"

สิ่งที่ผู้พูดควรพูด (~55 วินาที) "พอตรวจเสร็จ ข้อมูลถูกแยกเป็นสองกอง แยกที่ผ่านไปตารางที่ใช้งาน แยกที่ไม่ผ่านไปตารางกักกันพร้อมเหตุผล ไม่ถูกลบ จุดที่ขอบอกตรง ๆ คือ คะแนนคุณภาพ ถูกคำนวณหลังโหลดแล้ว มันไม่ได้เป็นประตูดักหยุดการโหลด ประตูดักคือการแยกรายแยกก่อนเขียน ส่วนคะแนนใช้ส่งแจ้งเตือน ติดสถานะ และตัดสินใจว่าจะสร้างตารางสรุปหรือไม่ ใครที่จะนำข้อมูลไปใช้ต่อ สามารถเรียก Trust-check เพื่อถามว่าตารางนี้ปลอดภัยพอหรือไม่ ซึ่งดูจากคะแนนล่าสุดและข้อเสนอเปลี่ยนโครงสร้างที่ยังค้างอยู่"

Technical evidence จากโค้ด

- ลำดับเขียน (Quarantine → MERGE → คิดคะแนน): SQE:1704-1735 , :1774 ; ลำดับ POST_LOAD: plan.py:5-6
- ผลของคะแนน: alert metrics.py:177-183 ; state warnings report.py:75 ; Gold ถูก trigger เมื่อ score ≥ threshold SQE:1777-1788 (subprocess ไม่ตรวจผล); AI advisor เมื่อ score < threshold – 15 advisory.py:44-46
- Threshold: dynamic_rules_engine.py:249-335 (ต้องมี ≥ 2 รอบก่อนใช้แบบ adaptive)
- Trust-check: API/lineage.py:340-447 — is_safe_to_consume = score ≥ threshold และไม่มี proposal ค้าง; ไม่พบ run → HALT_INGEST (:367-373)
- เงื่อนไขที่หยุดก่อนโหลดจริง: ไฟล์ว่าง (exit 75 → SKIPPED), ตารางถูกล็อกโดยรอบอื่น (SKIPPED), คีย์ที่ลงทะเบียนไว้ขาดในหัวไฟล์ (API 400)
- ข้อมูลเสียหลังล้มเหลว: MERGE ล้ม → fail-closed ไม่แตะ Active (SQE:1737-1753) แต่ Quarantine ที่เขียนแล้วคงอยู่

ประโยคจำ: แยกก่อนโหลด · วัดหลังโหลด · ไม่ทิ้งอะไรเลย **ประโยคเชื่อมโยง:** "แล้วข้อมูลที่ผ่านแล้วถูกเก็บไว้ที่ไหน?"

ครอบคลุมเกณฑ์: ข้อ 4 (การแยก valid/invalid) · ข้อ 5 (เส้นทางไปปลายทาง) · ข้อ 2

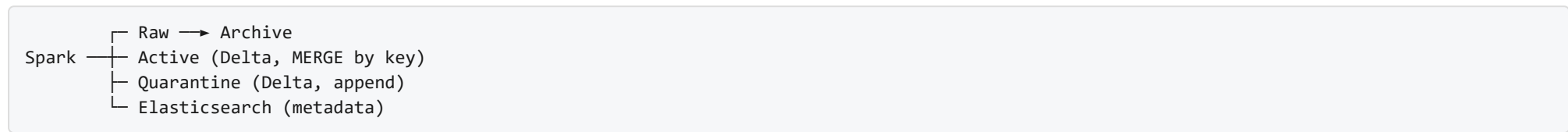
Slide 7 — "ข้อมูลถูกจัดเก็บที่ไหน?"

Key Message: ข้อมูลแต่ละประเภทมีที่อยู่ของมัน: ดิบ / ใช้งาน / กักกัน / ประวัติการรัน — และรันซ้ำไม่ทำให้ตารางใช้งานซ้ำซ้อน

เนื้อหาบนสไลด์ (4 โชน สไลด์การ์ดสีของหน้า 2 ในไฟล์อ้างอิง)

- **01 Raw → Archive** ● — `/data/raw/<ตาราง>/<ingest_id>` เก็บไว้รันซ้ำได้ แล้วย้ายไป `/data/archive` เมื่อสำเร็จ
- **02 Active (Delta)** ● — แกลว์ที่ผ่าน เขียนแบบ **MERGE ตามคีย์หลัก** (มีแล้วอัปเดต ไม่มีแล้วเพิ่ม)
- **03 Quarantine (Delta)** ● — แกลว์ไม่ผ่าน เขียนต่อท้าย แบ่งตาม `run_id` พร้อมเหตุผล
- **04 ประวัติ/Metadata → Elasticsearch** ● — สถานะ run, คะแนน, จำนวนแกลว์, lineage ระดับตาราง, ทะเบียน schema และกฎ
- (เส้นแยก) ● Reddit stream → Parquet + ES ไม่ผ่านโซนข้างบน
- กล่อง "สิ่งที่ระบบปรับปรุงกัน": **MERGE ล้มเหลว** → **ไม่แตะตาราง Active** · Quarantine ถูกลบตาม `ingest_id` ก่อนเขียนใหม่ (รันซ้ำไม่ซ้ำ)
- กล่อง "ข้อจำกัดที่รู้": ● ไม่มี rollback · ● ไม่เทียบจำนวนต้นทางกับปลายทาง · ● ไม่มี incremental จากต้นทาง (CDC) · ● ไม่มี ES bulk/retry
- แถบสรุปล่าง: "เก็บทั้งของดี ของเสีย และประวัติ — แยกที่กันชัดเจน"

Diagram ที่ควรรวาท — โซนสี่ 4 โซนบน HDFS + Elasticsearch ด้านข้าง



ลูกศรเลขกำกับลำดับ: ① Quarantine ② MERGE Active ③ Metadata ④ Archive

สิ่งที่ผู้พูดควรพูด (~50 วินาที) "ข้อมูลที่ผ่านแล้วถูกเขียนเข้า Delta Lake ตารางใช้งาน ด้วยการ MERGE ตามคีย์หลัก ถ้ารันไฟล์เดิมซ้ำ แกลว์เดิมถูกอัปเดต ไม่ซ้ำ แกลว์ที่ไม่ผ่านถูกเขียนเข้า ตารางกักกัน เราเขียนตารางกักกันก่อน แล้วค่อย MERGE ถ้า MERGE ล้มเหลว ระบบจะไม่แตะตารางใช้งาน ไฟล์ดิบถูกย้ายไป Archive และประวัติการรันทั้งหมดเก็บใน Elasticsearch ข้อจำกัดที่ต้องบอกคือยังไม่มี rollback ยังไม่มีการเทียบจำนวนแกลว์ต้นทางกับปลายทาง และยังไม่มีการไหลแบบเฉพาะส่วนที่เปลี่ยนจากฝั่งต้นทาง"

Technical evidence จากโค้ด

- Quarantine: `SQE:1704-1711` (ลบแกลว์ `ingest_id` เดิม :1706-1709; `append, partitionBy("run_id"), mergeSchema=true`)
- Active: `SQE:1713-1735` (รอบแรก `overwrite` :1730-1736; รอบต่อไป `MERGE ... whenMatchedUpdateAll().whenNotMatchedInsertAll()` ตามคีย์ :1718-1729)
- fail-closed: `SQE:1737-1753` · `autoMerge :85` · `OPTIMIZE/VACUUM` ทุก 10 เวอร์ชัน :1756-1765 (● อาจไม่ทำงานถ้าไม่มีคอลัมน์ `row_hash` — อนุมานจากโค้ด)
- Archive: `SQE:1790-1805` · (เส้นทางเก่า/n8n ที่ไม่มี `ingest_id` ลบ raw ทั้งหมดเมื่อไม่มี quarantine :1798-1803)
- ES: `report.py:52-80, run_registry.py:50-68`; เขียนทีละเอกสาร ไม่ retry (`SQE:96-121`) — ถ้าล้มเหลวแค่ print ทำให้ run ยังขึ้น SUCCEEDED (●)
- คีย์หลักที่อนุมานเอง: `S/common/keys.py:14-55` (ไม่ซ้ำ ≥ 90%; ไม่เจอ → `row_hash`)
- สิ่งที่ต้องรู้ถ้าถูกถามเรื่อง "ครบไหม": `total_records = clean + quarantine` (`assembly.py:45`) จึงลงตัวเสมอ ไม่ใช่การเทียบกับต้นทาง; แกลว์ที่ถูกลบซ้ำชั้นแรกไม่ถูกนับ
- ไม่พบ: `_bulk, .jdbc(, saveAsTable, rollback/RESTORE` (grep ทั่ว repo); Postgres ไม่ใช่ปลายทาง

ประโยคจำ: ของต้อยู่ที่หนึ่ง ของเสียอยู่อีกที่ ประวัติดูอีกที่ **ประโยคเชื่อม:** "เก็บแล้ว — แล้วนำไปใช้ทำอะไรได้บ้าง?"

ครอบคลุมเกณฑ์: ข้อ 5 (หลัก)

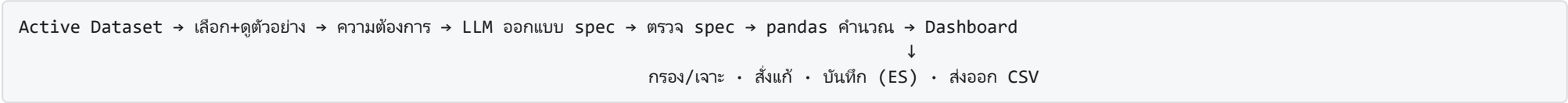
Slide 8 — "ข้อมูลที่ผ่านแล้ว นำไปใช้ทำอะไร?"

Key Message: ปลายทางของท่อคือคนที่ต้องตัดสินใจ — ระบบมีทั้งสร้างแดชบอร์ดด้วยตัวเอง ส่งออกข้อมูล และตรวจสอบว่าข้อมูลใช้ได้หรือไม่

เนื้อหาบนสไลด์

- **ฟีเจอร์หลัก: สร้างแดชบอร์ดจากข้อมูลที่ผ่านแล้ว** ● 1. เลือกชุดข้อมูล (จาก Active) · 2. ดูตัวอย่าง 20 แถว · 3. บอกความต้องการเป็นภาษาธรรมชาติ · 4. AI ออกแบบโครงสร้าง · 5. ระบบตรวจโครงสร้าง · 6. คำนวณตัวเลขจริง · 7. กรอง/คลิกเจาะ/สั่งแก้ · 8. บันทึกและเปิดเข้าได้
- **กล่องเน้น: AI ออกแบบ "โครง" ส่วน "ตัวเลข" คำนวณด้วย pandas** · AI ไม่เห็นแถวข้อมูล เห็นแค่ชื่อคอลัมน์และสถิติสรุป
- **ใช้งานอื่น:** ● ส่งออก CSV (active / quarantine / raw / gold) · ● Trust-check · ● อนุมัติ schema / กฎ · ● หน้า Dashboard และ Analytics (มีตัวเลขจริงจาก ES + บางส่วนจำลอง — ดูกล่อง)
- **กล่อง "ส่วนที่ไม่ใช่ของจริง ขอบอกก่อน":** ● กราฟ Sell-In/Sell-Out · ● ข้อความเล่าเรื่องธุรกิจบางส่วน · ● ปุ่ม "นำไปใช้" ของ Recommendation · ● ไม่มี Grafana dashboard · ● ไม่มี Prometheus ที่รันอยู่
- **แถบสรุปล่าง: "ข้อมูลที่เก็บไว้เฉย ๆ ไม่สร้างคุณค่า — ท่อต้องจบที่การใช้งาน"**

Diagram ที่ควรรวาท (สไลด์สายพานแนวนอน)



ใต้กล่อง LLM เขียน "ไม่คำนวณตัวเลข" ใต้ pandas เขียน "คำนวณจริง"

สิ่งที่ผู้พูดควรพูด (~60 วินาที) "ข้อมูลที่ผ่านท่อมาแล้วไม่ควรจบที่การเก็บ ระบบของเรามีฟีเจอร์สร้างแดชบอร์ด ผู้ใช้เลือกชุดข้อมูลผ่านการตรวจแล้ว พิมพ์ความต้องการ เช่น อยากเห็นยอดขายรายเดือน AI จะออกแบบโครงสร้างแดชบอร์ด แต่ AI ไม่ได้คำนวณตัวเลขเอง และไม่เห็นข้อมูลรายแถว มันเห็นแค่ชื่อคอลัมน์กับสถิติสรุป โครงสร้างที่ได้ต้องผ่านการตรวจกับรายการที่อนุญาต แล้ว pandas เป็นผู้คำนวณตัวเลขจริง ผู้ใช้ตรวจเทียบได้จากตัวอย่างแถวและตัวนับจำนวนแถว นอกจากนี้ยังส่งออกเป็น CSV ได้ และเรียก Trust-check เพื่อดูว่าตารางนี้ใช้ได้ไหม ขอบอกตรง ๆ ว่าหน้า Dashboard หลักของระบบบางส่วนเป็นข้อมูลจำลอง เช่นกราฟ Sell-In Sell-Out ซึ่งเราไม่ได้นำเสนอเป็นผลจริง"

Technical evidence จากโค้ด

- เส้นทางทั้งหมด: `API/dashboards.py:73-185` · ตัวอย่าง/profile `dashboard_data.py:97-233` · โหลดตาราง Active ทั้งก่อนเข้า API ผ่าน WebHDFS+pandas (cache 120 วินาที ไม่เกิน 4 เฟรม) :157-171
- ตรวจสอบ spec: `dashboard_spec.py:191-216` — whitelist ชนิด widget/aggregation, ไม่เกิน 16 widget / 6 filter, ตัดคอลัมน์ที่ไม่มีจริง; ไม่รันโค้ดหรือ SQL จาก LLM (:1-6)
- คำนวณ: `dashboard_compute.py:97-222` (KPI/bar/line/area/pie/table, filter, drill) — LLM ไม่สร้างตัวเลข
- บันทึก: `ES_sdoqap_dashboards` เก็บ spec ไม่ใช่ตัวเลข เปิดเข้าจึงคำนวณใหม่จากข้อมูลปัจจุบัน (`dashboards.py:137-185`)
- Refine: `dashboards.py:94-105` ต้องใช้ LLM (ไม่มี fallback แบบกฎ); แก้ได้ไม่เกิน 50 ครั้ง
- ส่งออก: `API/data_export.py:552-685` · Trust-check: `API/lineage.py:340-447`
- ส่วนจริงของหน้า Dashboard: KPI สุขภาพข้อมูล/SLA/freshness/COPDQ จาก ES `analytics.py:12-377, 673-766`; forecast (linear regression) :460-597
- ส่วนจำลอง/ไม่จริง: Sell-In/Out `analytics.py:768-831` (มี TODO ในโค้ดและ `is_example: True`); ข้อความ narrative/ระยะเวลาปัญหา :176-301; แขนเวลากราฟ anomaly ถูกสร้างขึ้น :383-387; ปุ่ม "นำไปใช้" `Analytics.jsx:32-41`; metadata ของ lineage node `lineage.py:156-301`
- Grafana: มี datasource + alert rules เท่านั้น (`infra/grafana/provisioning/` ไม่มี dashboard) · Prometheus: มีไฟล์ config แต่ไม่มี service ใน compose
- ความปลอดภัย (ถ้าถูกถาม): route export/analytics/gold/quality ไม่พบการบังคับล็อกอิน; Dashboard Builder บังคับ `require_session`

ประโยคจ่า: ข้อมูลที่ดีที่ไม่มีคนใช้ ก็ไม่ต่างจากไม่มีข้อมูล **ประโยคเชื่อม:** "ในเรื่องทั้งหมดนี้ AI เข้ามาช่วยตรงไหนกันแน่?"

ครอบคลุมเกณฑ์: ข้อ 6 (หลัก) · เกี่ยวกับข้อ 5 (ข้อมูลปลายทางที่ถูกใช้)

Slide 9 — "AI / LLM ช่วยตรงไหน?" (พื้นมืด — อิงรูปแบบหน้า 5-6 ของไฟล์อ้างอิง)

Key Message: AI ในระบบนี้ช่วย 4 งานที่เจาะจง — ส่วนตรวจสอบคุณภาพ แปลง และคำนวณตัวเลข เป็นกฎและสถิติที่ตรวจสอบได้ ไม่ใช่ AI

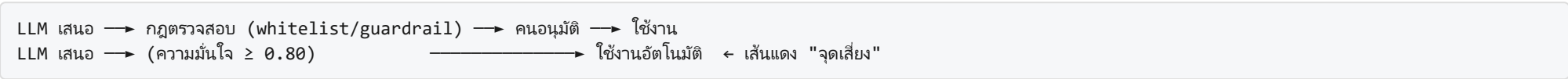
เนื้อหาบนสไลด์

- การ์ดเลข 01-04: AI ช่วยตรงนี้จริง
 - **01 ออกแบบแดชบอร์ด** 🟢 — LLM (Groq) สร้างโครง (spec) จากความต้องการ + สถิติคอลัมน์; ระบบตรวจก่อนใช้ ถ้า LLM ใช้ไม่ได้ → โหมดกฎอัตโนมัติ
 - **02 เสนอกฎคุณภาพ** 🟢 — AI advisor วิเคราะห์ตัวอย่างแถวที่ถูกกัก (≤ 20 แถว ปิดคอลัมน์ที่เป็นรหัสประจำตัว) → เป็น "ข้อเสนอ" ต้องมีคนอนุมัติ
 - **03 สร้างกฎแก้ไขอัตโนมัติ** 🟡 — เมื่อมีแถวถูกกัก ระบบให้ LLM สร้างกฎแก้ แล้ว **ใช้อัตโนมัติถ้าความมั่นใจ ≥ 0.80** และรันซ้ำหนึ่งรอบ (ไม่ต้องมีคนอนุมัติ)
 - **04 เรียบเรียงข้อความสรุปบนหน้า Dashboard** 🟡 — ใช้ข้อเท็จจริงที่คำนวณไว้แล้ว ไม่คิดตัวเลขใหม่
- ตารางเทียบกับ 5 จุดที่ไฟล์อ้างอิงให้ AI ช่วย (พูดตรง ๆ ว่าอะไรคือ AI อะไรไม่ใช่):

จุดที่ AI ETL หัวไปกล่าวถึง	SDOQAP ทำอย่างไรจริง
Intelligent Schema Mapping	ไม่ใช่ AI — จัดชื่อด้วยอัลกอริทึม (normalize_name) + ตรวจสอบ drift แล้วให้คนอนุมัติ ●
Dynamic Transformation	กฎ DSL (fillna/calculate/cast/filter/standardize) — ตั้งโดยผู้ใช้ หรือ LLM สร้างในโหมดแก้อัตโนมัติ ●
Automated Anomaly Detection	สถิติ + ML ไม่ใช่ LLM — IQR, Z-score 3.0, และ decision tree (Spark ML) เรียนรู้กฎจากแถวที่ผ่าน/ไม่ผ่าน (ต้องมีคนอนุมัติ) ●
Automated Remediation	LLM สร้างกฎแก้ + ใช้เมื่อมั่นใจ ≥ 0.80 และรันซ้ำ 1 รอบ ● · ผลการแก้ไขไม่ได้ถูกตรวจสอบซ้ำ (จำนวน "fixed" = จำนวนแถวในกลุ่ม)
AI Dashboard & Analytics	LLM ออกแบบ spec ● · pandas คำนวณ · forecast ใช้ linear regression ไม่ใช่ AI

- กล้อง "ไม่ใช่ AI": คะแนนคุณภาพ · การตรวจกฎทั้งหมด · IQR/Z-score · ตัวเลข KPI และกราฟ · alert · forecast
- แถบสรุปล่าง: "AI ช่วยเสนอและออกแบบ — ตัวเลขและการตัดสินใจคุณภาพมาจากกฎที่ตรวจสอบย้อนได้"

Diagram ที่ควรรวด — แกนตั้ง 3 ระดับ "ใครตัดสินใจ" (เหมือนกรวยความเชื่อมั่น)



สิ่งที่ผู้พูดควรพูด (~60 วินาที) "AI ในระบบเรามี 4 งานเฉพาะ งานแรกคือช่วยออกแบบแดชบอร์ด งานที่สองคือเสนอกฎคุณภาพจากตัวอย่างแถวที่ถูกกัก โดยกฎเหล่านี้เป็นข้อเสนอ ต้องมีคนอนุมัติและมีเพดานป้องกัน เช่น เกณฑ์คะแนนห้ามต่ำกว่า 70 งานที่สามคือสร้างกฎแก้ไขอัตโนมัติ ซึ่งเป็นจุดเดียวที่ AI ปรับกฎได้โดยไม่ต้องรอคน ถ้าความมั่นใจถึง 0.80 และงานที่สี่คือเรียบเรียงข้อความบนหน้า Dashboard ส่วนการตรวจคุณภาพ การแปลง และการคำนวณตัวเลข เป็นกฎและสถิติ ไม่ใช่ AI เราอยากให้มีคุณภาพตรวจสอบย้อนกลับได้ ถ้าถามว่าส่งข้อมูลอะไรให้ AI ส่วนแดชบอร์ดส่งแค่สถิติคอลัมน์ ไม่มีแถวข้อมูล แต่ตัวแก้อัตโนมัติส่งตัวอย่างแถวที่ถูกกัก 5 แถวไปโดยไม่ปิดบัง ซึ่งเป็นข้อควรระวัง"

Technical evidence จากโค้ด

- Dashboard LLM: API/dashboard_llm.py — Groq openai/gpt-oss-120b (หรือค่าที่บันทึกไว้), temp 0.2, timeout 20 วินาที, retry 1 ครั้ง (เฉพาะเมื่อคำตอบแรกมาใน 25 วินาที :25-26,156-173); ส่งเฉพาะ profile (profile_for_prompt :120-130; คอลัมน์ที่ดูเป็น id ไม่ส่ง min/max :110-117); fallback :176-209
- AI advisor: ai_rule_advisor.py, S/stages/advisory.py:6-167 — Groq → Ollama qwen2.5:3b → heuristic; ตัวอย่างสูงสุด 20 แถว (:285) ปิดคอลัมน์ identifier (_redact_identifiers :260-274); เก็บเป็น PROPOSED ใน ES (:1079); เปิดโดยค่าเริ่มต้น ai_advisor.enabled=true, trigger="always" (rules_config.json:42-48)
- กฎการอนุมัติ: API/dynamic_rules.py:564-594 (พื้น 70, ลดไม่เกิน 10%, ห้ามปิด critical checks, null tolerance ≤ 0.30)
- แก้ไขอัตโนมัติ: auto_remediation_engine.py:233-339 — Ollama ก่อน แล้ว Groq; ตัวอย่าง 5 แถว ไม่ปิดบัง (:238); REMEDIATION_CONFIDENCE 0.80 (:154, 331); เขียนกฎลง registry (:27-96); daemon รันซ้ำหนึ่งรอบ/ตาราง (spark_trigger_daemon.py:137-179)

- Decision tree: `ai_rule_advisor.py:825-985` (Spark ML) — ● เรียนจากป้ายผ่าน/ไม่ผ่านของระบบเอง (วนกลับ)
- ข้อความ AI บน Dashboard: `whitebox.py:1774-1856` — แสดงป้าย "โดย AI" เฉพาะเมื่อสร้างจริง
- ข้อควรรู้: ผลของ LLM รูปแบบ `suggested_rules` ไม่ตรงกับที่ชั้นอนุมัติ merge (ต้องมี `rule_path`) จึงอนุมัติแล้วอาจไม่เปลี่ยน `rules_config.json` ●
(`dynamic_rules.py:553-621`); Ollama ผูก profile `ai` และไม่พบสคริปต์ pull โมเดล — ยังไม่ยืนยันว่าพร้อมใช้ตอนรัน
- ไม่ใช่ LLM: `metrics.py` (score, alert), `analytics.py` (forecast, clustering, impact), `dashboard_compute.py`

ประโยคจำ: AI เสนอ — กฎตัดสิน — คนอนุมัติ (ยกเว้นตัวแก้อัตโนมัติ) **ประโยคเชื่อม:** "ทั้งหมดนี้ต่อกันเป็นระบบเดียวอย่างไร?"

ครอบคลุมเกณฑ์: ข้อ 6 (AI ช่วยการใช้ประโยชน์) · ข้อ 4 (กฎ/การแก้ข้อมูลที่ AI มีส่วน) · ข้อ 2 (กระบวนการนอก stage: advisor, auto-remediation)

Slide 10 — "ภาพรวมทั้งระบบ ตั้งแต่ต้นจนจบ"

Key Message: SDOQAP ไม่ได้ทำแค่ ETL แต่รวมการตรวจคุณภาพ การกักกัน การติดตาม และการนำข้อมูลไปใช้ ไว้ในวงจรเดียว

เนื้อหาบนสไลด์ (เรียงลงแนวนั่ง 7 ชั้น แต่ละชั้น 1 ประโยค พร้อมจุดสถานะ)

1. แหล่งข้อมูล — ไฟล์ CSV/Excel · REST API · PostgreSQL ● (Reddit stream แยกเส้น ●)
2. รับเข้า — ตรวจสอบสิทธิ์ ชื่อ ขนาด ไฟล์ว่าง ไฟล์ซ้ำ แล้วเก็บดิบใน HDFS ต่อหนึ่งการนำเข้า ●
3. แปลง — Spark จัดโครงสร้าง ล้าง ลบซ้ำ มาตรฐานค่า ผ่าน 21 ชั้น ●
4. ตรวจคุณภาพ — กฎ + สถิติ ระบุเหตุผลทุกแถวที่ไม่ผ่าน ●
5. ผ่าน / กักกัน — แถวผ่านไป Active, แถวไม่ผ่านไป Quarantine; คะแนนคิดหลังโหลดเพื่อเตือน ●
6. จัดเก็บ — Delta MERGE ตามคีย์ + ประวัติใน Elasticsearch + archive ไฟล์ดิบ ● (ยังไม่มี rollback/กระหายอดต้นทาง)
7. ใช้ประโยชน์ — สร้างแดชบอร์ดด้วย AI · ส่งออก · Trust-check ● (บางส่วนของหน้า Dashboard เป็นข้อมูลจำลอง ●) - แถบปิดท้าย (ตัวใหญ่): **"SDOQAP ไม่ได้ทำแค่ ETL แต่รวม Data Quality, Quarantine, Monitoring และการนำข้อมูลไปใช้ผ่าน Dashboard เข้าด้วยกัน"** - มุมล่างขวา: เส้นแยก Reddit → Kafka → Spark Streaming → Parquet + ES (ยังไม่เข้าระบบตรวจ) - ป้าย "Monitoring" ●: เก็บเมตริกและส่ง alert ได้ในโค้ด แต่ปลายทางแจ้งเตือน (Slack/LINE) ต้องตั้งค่าเอง, workflow n8n ต้อง import เอง, Grafana มี datasource + alert rules แต่ไม่มี dashboard

Diagram ที่ควรรว — ภาพเดียวแนวนอน 7 กล้อง (ต่อกันด้วยลูกศรหนา เหมือนหน้า 2 ของไฟล์อ้างอิง) แถวล่าง = ที่เก็บ (HDFS · Delta · Elasticsearch) และแถบน = ผู้กระทำ (API · Spark · LLM · ผู้ใช้)

แหล่งข้อมูล → รับเข้า → แปลง → ตรวจสอบภาพ → ผ่าน/กักกัน → จัดเก็บ → ใช้ประโยชน์
(ผู้ใช้) (API) (Spark) (Spark) (Spark) (Delta+ES) (ผู้ใช้+LLM+pandas)

สิ่งที่ผู้พูดควรพูด (~50 วินาที) "สรุปภาพรวม ข้อมูลเข้ามาจากไฟล์ API หรือฐานข้อมูล ผ่านด้านตรวจที่ API เก็บดิบใน HDFS แล้ว Spark แปลงและตรวจผ่าน 21 ชั้น แยกที่ผ่านไปตารางใช้งานด้วย MERGE แยกที่ไม่ผ่านไปตารางกักกันพร้อมเหตุผล ประวัติถูกรอบเก็บใน Elasticsearch จากนั้นผู้ใช้ใช้ข้อมูลที่ผ่านแล้วสร้างแดชบอร์ดโดยมี AI ช่วยออกแบบ หรือส่งออกเป็น CSV ข้อจำกัดที่เราเจอและเปิดเผยคือ ยังไม่มี rollback ยังไม่กระหนบยอดต้นทางกับปลายทาง คะแนนคุณภาพคิดหลังโหลด และบางส่วนของหน้า Dashboard เป็นข้อมูลจำลอง สรุปคือ SDOQAP ไม่ได้ทำแค่ ETL แต่รวมการตรวจสอบภาพ การกักกัน การติดตาม และการนำข้อมูลไปใช้เข้าด้วยกัน"

Technical evidence จากโค้ด

- ลำดับงานทั้งรอบ: SQE:run_quality_check (spark_quality_engine.py:1551-1811); จุดเชื่อม API → daemon: API/pipeline.py:230-242 ; คิว/สถานะ: trigger_core.py:45-82
- Workflow จริงฉบับเต็ม: [system-actual-analysis-topics-2-6.md](#) หัวข้อ 7
- Stack: docker-compose.yml (profile streaming,ai,tools); n8n infra/n8n/ingestion_workflow.json (ตั้งเวลา 30 นาที, active: true :863)
- Alert: Spark send_n8n_alert (SQE:1466-1493) → alert_router.py (ไม่ตั้ง SLACK_WEBHOOK_URL / LINE_NOTIFY_TOKEN = พิมพ์ log อย่างเดียว :83-88); Grafana rules infra/grafana/provisioning/alerting/alert_rules.yaml (กฎที่ 2 ข้อความบอกอัตรา แต่คำนวณเฉลี่ยจำนวน)
- ยังไม่ได้ยืนยัน (ต้องรันระบบจริง): พฤติกรรม runtime ทั้งหมด, ผลทดสอบ, scalability/HA

ประโยคจ่า: ไม่ใช่แค่ ETL — แต่เป็นวงจรตั้งแต่รับข้อมูลจนถึงใช้ข้อมูล ประโยคเชื่อม (ปิดการนำเสนอ): "ขอบคุณครับ/ค่ะ — ยินดีรับคำถาม"

ครอบคลุมเกณฑ์: ข้อ 2, 3, 4, 5, 6 (ครบทั้งหมด ในภาพเดียว)

ตารางตรวจสอบความครอบคลุมเกณฑ์ (ใช้ตรวจสอบ ไม่ต้องขึ้นสไลด์)

สไลด์	ข้อ 2 กระบวนการ	ข้อ 3 Extraction	ข้อ 4 Transformation	ข้อ 5 Loading	ข้อ 6 Utilization
1 ข้อมูลมาจากไหน	บริบท	● (แหล่งที่รองรับ)			
2 ทำไมหลายกระบวนการ	●		บริบท		
3 ก่อนรับเข้า	○	●			
4 ทำให้ใช้ร่วมกันได้	○		●		

สไลด์	ข้อ 2 กระบวนการ	ข้อ 3 Extraction	ข้อ 4 Transformation	ข้อ 5 Loading	ข้อ 6 Utilization
5 ตรวจสอบคุณภาพ	○		●		
6 ผ่าน/ไม่ผ่าน	○		● (แยก valid/invalid)	● (เส้นทางไปปลายทาง)	
7 จัดเก็บที่ไหน				●	
8 นำไปใช้				บริบท	●
9 AI/LLM	○		○		●
10 End-to-End	●	●	●	●	●

● = หลัก ○ = เกี่ยวข้อง

Cheat Sheet (1 หน้า) — ซ่อมพุดตามเรื่อง

#	ข้อสไลด์	ใจความ 1 ประโยค	ตัวเลข/ค่าที่ต้องจำ	ห้ามพุดเกินจริง
1	ข้อมูลมาจากไหน?	4 ทางเข้า: ไฟล์ · REST · PostgreSQL · stream (stream เป็นเส้นแยก)	3 ทางต่อท่อครบ	เทียบ API/Stream ใน UI เป็นโหมดสาธิต
2	ทำไมหลายกระบวนการ?	หนึ่งปัญหา หนึ่งกระบวนการ → 21 ชั้น	1 + 12 + 8 · inline 7	อย่าบอกว่าทุก stage ผลเท่ากัน (บางตัวผลน้อย)
3	ก่อนรับเข้า	ด้านตรวจ 7 ข้อ แล้วเก็บดิบต่อ 1 การนำเข้า	200 MB · SHA-256 · 1,000,000 แถว · retry 5	ยังไม่เทียบจำนวนแถวต้นทาง · ไม่ตรวจ encoding
4	ทำให้ใช้ร่วมกัน	อ่านเป็นข้อความก่อน แล้วแปลงตามโครงสร้าง	1,234→1234 · 22-7-2569→2026-07-22	rename คือ อัลกอริทึม ไม่ใช่ AI
5	ตรวจสอบคุณภาพ	3 ชั้น: กฎ · สถิติ · แนวโน้ม + เหตุผลทุกแถว	Z-score 3.0 · IQR 1.5 · score = Clean/(Clean+Q)×100	บางสวิตช์ใน config ยังไม่ถูกอ่าน
6	ผ่าน/ไม่ผ่าน	แยกรายแถวก่อนโหลด · คะแนนหลังโหลด	เกณฑ์ 90 / พื้น 70	คะแนนไม่หยุดการโหลด
7	จัดเก็บที่ไหน	Raw/Archive · Active · Quarantine · ES	MERGE ตามคีย์ · เขียน Quarantine ก่อน	ไม่มี rollback · ไม่กระหนบยอดต้นทาง · ไม่มี CDC

#	ชื่อสไลด์	ใจความ 1 ประโยค	ตัวเลข/ค่าที่ต้องจำ	ห้ามพูดเกินจริง
8	นำไปใช้	สร้างแดชบอร์ด AI จาก Active + ส่งออก + Trust-check	AI ออกแบบโครง · pandas คำนวณ	Sell-In/Out = จำลอง · ปุ่ม "นำไปใช้" ไม่มีผลจริง · ไม่มี Grafana dashboard
9	AI ช่วยตรงไหน	4 งาน: แดชบอร์ด · เสนอกฎ · แก้อัตโนมัตi · เรียบเรียงข้อความ	ความมั่นใจ ≥ 0.80 · พัน 70	ตรวจคุณภาพ/แปลง/ตัวเลข ไม่ใช่ AI · ตัวแก้อัตโนมัตiไม่ผ่านคนอนุมัติ
10	ภาพรวม	วงจรครบ 7 ชั้น ไม่ใช่แค่ ETL	—	monitoring ต้องตั้งค่าเพิ่ม (Slack/LINE/n8n)

ประโยคปิดที่ต้องจำ: "SDOQAP ไม่ได้ทำแค่ ETL แต่รวม Data Quality, Quarantine, Monitoring และการนำข้อมูลไปใช้ผ่าน Dashboard เข้าด้วยกัน"